Module 3: Numerical Data Analysis with NumPy

Learning Objectives

- Introduction to NumPy library
- Working with arrays and matrices
- Mathematical operations on arrays
- Array manipulation and indexing
- Statistical functions and operations

Topics Covered

1. Introduction to NumPy and its advantages
2. Creating NumPy arrays
3. Array attributes and properties
4. Array indexing and slicing
5. Array operations and broadcasting
6. Mathematical functions
7. Statistical operations
8. Linear algebra operations
9. Array reshaping and manipulation
10. Working with missing data
11. Performance comparison with Python lists

Exercises

- NumPy array creation and manipulation exercises
- Mathematical and statistical analysis problems

1. Introduction to NumPy and its Advantages

In [1]:
```
import numpy as np
import time

# NumPy advantages demonstration
print("NumPy Version:", np.__version__)

# Performance comparison: NumPy vs Python lists
size = 1000000

# Python list
python_list = list(range(size))
start_time = time.time()
python_squared = [x**2 for x in python_list]
```

```
python_time = time.time() - start_time

# NumPy array
numpy_array = np.arange(size)
start_time = time.time()
numpy_squared = numpy_array**2
numpy_time = time.time() - start_time

print(f"Python list time: {python_time:.6f} seconds")
print(f"NumPy array time: {numpy_time:.6f} seconds")
print(f"NumPy is {python_time/numpy_time:.2f}x faster!")

# Memory efficiency
import sys
python_memory = sys.getsizeof(python_list) + sys.getsizeof(python_squared
numpy_memory = numpy_array.nbytes + numpy_squared.nbytes

print(f"Python list memory: {python_memory} bytes")
print(f"NumPy array memory: {numpy_memory} bytes")
print(f"NumPy uses {numpy_memory/python_memory:.2f}x less memory!")
```

```
NumPy Version: 2.3.2
Python list time: 0.065569 seconds
NumPy array time: 0.005313 seconds
NumPy is 12.34x faster!
Python list memory: 16448784 bytes
NumPy array memory: 16000000 bytes
NumPy uses 0.97x less memory!
```

2. Creating NumPy Arrays

In [2]:

```
# Creating arrays from lists
list_data = [1, 2, 3, 4, 5]
arr1 = np.array(list_data)
print(f"From list: {arr1}")
print(f"Type: {type(arr1)}")
print(f"Data type: {arr1.dtype}")

# Creating 2D arrays
matrix_data = [[1, 2, 3], [4, 5, 6], [7, 8, 9]]
arr2d = np.array(matrix_data)
print(f"\n2D array:\n{arr2d}")
print(f"Shape: {arr2d.shape}")

# Creating arrays with specific data types
arr_int = np.array([1, 2, 3], dtype=np.int32)
arr_float = np.array([1.0, 2.0, 3.0], dtype=np.float64)
print(f"\nInteger array: {arr_int}, dtype: {arr_int.dtype}")
print(f"Float array: {arr_float}, dtype: {arr_float.dtype}")

# Array creation functions
zeros = np.zeros(5)
ones = np.ones((3, 4))
full = np.full((2, 3), 7)
print(f"\nZeros: {zeros}")
print(f"Ones:\n{ones}")
print(f"Full (7s):\n{full}")

# Range arrays
```

```python
range_arr = np.arange(0, 10, 2)
linspace_arr = np.linspace(0, 1, 5)
print(f"\nArange: {range_arr}")
print(f"Linspace: {linspace_arr}")

# Random arrays
random_arr = np.random.random((3, 3))
print(f"\nRandom array:\n{random_arr}")

# Identity matrix
identity = np.eye(3)
print(f"\nIdentity matrix:\n{identity}")

# Empty array (uninitialized)
empty_arr = np.empty((2, 3))
print(f"\nEmpty array:\n{empty_arr}")
```

```
From list: [1 2 3 4 5]
Type: <class 'numpy.ndarray'>
Data type: int64

2D array:
[[1 2 3]
 [4 5 6]
 [7 8 9]]
Shape: (3, 3)

Integer array: [1 2 3], dtype: int32
Float array: [1. 2. 3.], dtype: float64

Zeros: [0. 0. 0. 0. 0.]
Ones:
[[1. 1. 1. 1.]
 [1. 1. 1. 1.]
 [1. 1. 1. 1.]]
Full (7s):
[[7 7 7]
 [7 7 7]]

Arange: [0 2 4 6 8]
Linspace: [0.   0.25 0.5  0.75 1.  ]

Random array:
[[0.50029841 0.68356353 0.29554158]
 [0.19896512 0.08348556 0.36837322]
 [0.25030217 0.31072726 0.79165652]]

Identity matrix:
[[1. 0. 0.]
 [0. 1. 0.]
 [0. 0. 1.]]

Empty array:
[[0. 0. 0.]
 [0. 0. 0.]]
```

3. Array Attributes and Properties

In [3]:

```python
# Create a sample array
arr = np.array([[1, 2, 3, 4], [5, 6, 7, 8], [9, 10, 11, 12]])
print(f"Array:\n{arr}")

# Basic attributes
print(f"\nShape: {arr.shape}")
print(f"Number of dimensions: {arr.ndim}")
print(f"Size (total elements): {arr.size}")
print(f"Data type: {arr.dtype}")
print(f"Item size (bytes per element): {arr.itemsize}")
print(f"Total bytes: {arr.nbytes}")

# Array properties
print(f"\nIs contiguous: {arr.flags.c_contiguous}")
print(f"Memory layout: {arr.flags.f_contiguous}")

# Reshaping
reshaped = arr.reshape(2, 6)
print(f"\nReshaped to (2, 6):\n{reshaped}")

# Flattening
flattened = arr.flatten()
print(f"Flattened: {flattened}")

# Transpose
transposed = arr.T
print(f"\nTransposed:\n{transposed}")

# Array statistics
print(f"\nMin value: {arr.min()}")
print(f"Max value: {arr.max()}")
print(f"Mean: {arr.mean():.2f}")
print(f"Sum: {arr.sum()}")
print(f"Standard deviation: {arr.std():.2f}")

# Axis-wise operations
print(f"\nSum along axis 0 (columns): {arr.sum(axis=0)}")
print(f"Sum along axis 1 (rows): {arr.sum(axis=1)}")
print(f"Mean along axis 0: {arr.mean(axis=0)}")
print(f"Mean along axis 1: {arr.mean(axis=1)}")
```

```
Array:
[[ 1  2  3  4]
 [ 5  6  7  8]
 [ 9 10 11 12]]

Shape: (3, 4)
Number of dimensions: 2
Size (total elements): 12
Data type: int64
Item size (bytes per element): 8
Total bytes: 96

Is contiguous: True
Memory layout: False

Reshaped to (2, 6):
[[ 1  2  3  4  5  6]
 [ 7  8  9 10 11 12]]
Flattened: [ 1  2  3  4  5  6  7  8  9 10 11 12]

Transposed:
[[ 1  5  9]
 [ 2  6 10]
 [ 3  7 11]
 [ 4  8 12]]

Min value: 1
Max value: 12
Mean: 6.50
Sum: 78
Standard deviation: 3.45

Sum along axis 0 (columns): [15 18 21 24]
Sum along axis 1 (rows): [10 26 42]
Mean along axis 0: [5. 6. 7. 8.]
Mean along axis 1: [ 2.5  6.5 10.5]
```

4. Array Indexing and Slicing

In [4]:

```python
# Create a sample array
arr = np.array([[1, 2, 3, 4], [5, 6, 7, 8], [9, 10, 11, 12]])
print(f"Original array:\n{arr}")

# Basic indexing
print(f"\nElement at [0, 0]: {arr[0, 0]}")
print(f"Element at [2, 3]: {arr[2, 3]}")

# Slicing
print(f"\nFirst row: {arr[0, :]}")
print(f"Second column: {arr[:, 1]}")
print(f"First two rows: \n{arr[:2, :]}")
print(f"Last two columns: \n{arr[:, -2:]}")

# Step slicing
print(f"\nEvery other element in first row: {arr[0, ::2]}")
print(f"Reverse first row: {arr[0, ::-1]}")

# Boolean indexing
mask = arr > 5
```

```
print(f"\nBoolean mask (elements > 5):\n{mask}")
print(f"Elements > 5: {arr[mask]}")

# Fancy indexing
indices = [0, 2]
print(f"\nRows at indices {indices}:\n{arr[indices, :]}")

# Conditional indexing
even_mask = arr % 2 == 0
print(f"\nEven elements: {arr[even_mask]}")

# Modifying arrays
arr_copy = arr.copy()
arr_copy[0, 0] = 99
print(f"\nModified array:\n{arr_copy}")

# 3D array indexing
arr_3d = np.array([[[1, 2], [3, 4]], [[5, 6], [7, 8]]])
print(f"\n3D array:\n{arr_3d}")
print(f"Shape: {arr_3d.shape}")
print(f"Element at [1, 0, 1]: {arr_3d[1, 0, 1]}")
print(f"First 'layer': \n{arr_3d[0, :, :]}")
```

```
Original array:
[[ 1  2  3  4]
 [ 5  6  7  8]
 [ 9 10 11 12]]

Element at [0, 0]: 1
Element at [2, 3]: 12

First row: [1 2 3 4]
Second column: [ 2  6 10]
First two rows:
[[1 2 3 4]
 [5 6 7 8]]
Last two columns:
[[ 3  4]
 [ 7  8]
 [11 12]]

Every other element in first row: [1 3]
Reverse first row: [4 3 2 1]

Boolean mask (elements > 5):
[[False False False False]
 [False  True  True  True]
 [ True  True  True  True]]
Elements > 5: [ 6  7  8  9 10 11 12]

Rows at indices [0, 2]:
[[ 1  2  3  4]
 [ 9 10 11 12]]

Even elements: [ 2  4  6  8 10 12]

Modified array:
[[99  2  3  4]
 [ 5  6  7  8]
 [ 9 10 11 12]]

3D array:
[[[1 2]
  [3 4]]

 [[5 6]
  [7 8]]]
Shape: (2, 2, 2)
Element at [1, 0, 1]: 6
First 'layer':
[[1 2]
 [3 4]]
```

5. Array Operations and Broadcasting

In [5]:
```python
# Element-wise operations
arr1 = np.array([1, 2, 3, 4])
arr2 = np.array([5, 6, 7, 8])

print(f"Array 1: {arr1}")
print(f"Array 2: {arr2}")
```

```python
# Arithmetic operations
print(f"\nAddition: {arr1 + arr2}")
print(f"Subtraction: {arr1 - arr2}")
print(f"Multiplication: {arr1 * arr2}")
print(f"Division: {arr1 / arr2}")
print(f"Power: {arr1 ** 2}")

# Broadcasting examples
print(f"\n--- Broadcasting Examples ---")

# Scalar broadcasting
scalar = 10
print(f"Array + scalar: {arr1 + scalar}")
print(f"Array * scalar: {arr1 * scalar}")

# Different shapes
arr_2d = np.array([[1, 2, 3], [4, 5, 6]])
arr_1d = np.array([10, 20, 30])

print(f"\n2D array:\n{arr_2d}")
print(f"1D array: {arr_1d}")
print(f"2D + 1D (broadcasting):\n{arr_2d + arr_1d}")

# More broadcasting examples
arr_col = np.array([[1], [2], [3]])
arr_row = np.array([[10, 20, 30]])

print(f"\nColumn array:\n{arr_col}")
print(f"Row array: {arr_row}")
print(f"Column + Row (broadcasting):\n{arr_col + arr_row}")

# Comparison operations
print(f"\n--- Comparison Operations ---")
print(f"arr1 > 2: {arr1 > 2}")
print(f"arr1 == arr2: {arr1 == arr2}")
print(f"arr1 >= 3: {arr1 >= 3}")

# Logical operations
print(f"\n--- Logical Operations ---")
mask1 = arr1 > 2
mask2 = arr1 < 4
print(f"mask1 (arr1 > 2): {mask1}")
print(f"mask2 (arr1 < 4): {mask2}")
print(f"mask1 AND mask2: {mask1 & mask2}")
print(f"mask1 OR mask2: {mask1 | mask2}")
print(f"NOT mask1: {~mask1}")

# In-place operations
print(f"\n--- In-place Operations ---")
arr_original = np.array([1, 2, 3, 4])
arr_modified = arr_original.copy()
arr_modified += 10
print(f"Original: {arr_original}")
print(f"Modified (+= 10): {arr_modified}")

# Universal functions (ufuncs)
print(f"\n--- Universal Functions ---")
print(f"Square root: {np.sqrt(arr1)}")
print(f"Exponential: {np.exp(arr1)}")
print(f"Logarithm: {np.log(arr1)}")
```

```
print(f"Sine: {np.sin(arr1)}")
print(f"Absolute value: {np.abs(np.array([-1, -2, 3, -4]))}")
```

```
Array 1: [1 2 3 4]
Array 2: [5 6 7 8]

Addition: [ 6  8 10 12]
Subtraction: [-4 -4 -4 -4]
Multiplication: [ 5 12 21 32]
Division: [0.2        0.33333333 0.42857143 0.5       ]
Power: [ 1  4  9 16]

--- Broadcasting Examples ---
Array + scalar: [11 12 13 14]
Array * scalar: [10 20 30 40]

2D array:
[[1 2 3]
 [4 5 6]]
1D array: [10 20 30]
2D + 1D (broadcasting):
[[11 22 33]
 [14 25 36]]

Column array:
[[1]
 [2]
 [3]]
Row array: [[10 20 30]]
Column + Row (broadcasting):
[[11 21 31]
 [12 22 32]
 [13 23 33]]

--- Comparison Operations ---
arr1 > 2: [False False  True  True]
arr1 == arr2: [False False False False]
arr1 >= 3: [False False  True  True]

--- Logical Operations ---
mask1 (arr1 > 2): [False False  True  True]
mask2 (arr1 < 4): [ True  True  True False]
mask1 AND mask2: [False False  True False]
mask1 OR mask2: [ True  True  True  True]
NOT mask1: [ True  True False False]

--- In-place Operations ---
Original: [1 2 3 4]
Modified (+= 10): [11 12 13 14]

--- Universal Functions ---
Square root: [1.         1.41421356 1.73205081 2.        ]
Exponential: [ 2.71828183  7.3890561  20.08553692 54.59815003]
Logarithm: [0.         0.69314718 1.09861229 1.38629436]
Sine: [ 0.84147098  0.90929743  0.14112001 -0.7568025 ]
Absolute value: [1 2 3 4]
```

In [ ]:
```
## 6. Mathematical Functions

NumPy provides a comprehensive set of mathematical functions that operate
```

In [ ]:

```
# Mathematical Functions Examples

# Create sample arrays
arr = np.array([1, 2, 3, 4, 5])
arr_neg = np.array([-2, -1, 0, 1, 2])
arr_float = np.array([0.1, 0.5, 1.0, 1.5, 2.0])

print("=== BASIC MATHEMATICAL FUNCTIONS ===")
print(f"Original array: {arr}")

# Basic arithmetic functions
print(f"\nSquare root: {np.sqrt(arr)}")
print(f"Square: {np.square(arr)}")
print(f"Power of 3: {np.power(arr, 3)}")
print(f"Exponential: {np.exp(arr_float)}")
print(f"Natural logarithm: {np.log(arr)}")
print(f"Log base 10: {np.log10(arr)}")
print(f"Log base 2: {np.log2(arr)}")

# Trigonometric functions
angles = np.array([0, np.pi/6, np.pi/4, np.pi/3, np.pi/2])
print(f"\nAngles (radians): {angles}")
print(f"Sine: {np.sin(angles)}")
print(f"Cosine: {np.cos(angles)}")
print(f"Tangent: {np.tan(angles)}")
print(f"Arc sine: {np.arcsin(np.array([0, 0.5, 1]))}")
print(f"Arc cosine: {np.arccos(np.array([0, 0.5, 1]))}")
print(f"Arc tangent: {np.arctan(arr_float)}")

# Hyperbolic functions
print(f"\n=== HYPERBOLIC FUNCTIONS ===")
print(f"Hyperbolic sine: {np.sinh(arr_float)}")
print(f"Hyperbolic cosine: {np.cosh(arr_float)}")
print(f"Hyperbolic tangent: {np.tanh(arr_float)}")

# Rounding functions
arr_decimal = np.array([1.2, 2.7, 3.1, 4.9, 5.5])
print(f"\n=== ROUNDING FUNCTIONS ===")
print(f"Original: {arr_decimal}")
print(f"Round: {np.round(arr_decimal)}")
print(f"Floor: {np.floor(arr_decimal)}")
print(f"Ceiling: {np.ceil(arr_decimal)}")
print(f"Truncate: {np.trunc(arr_decimal)}")

# Absolute value and sign
print(f"\n=== ABSOLUTE VALUE AND SIGN ===")
print(f"Array with negatives: {arr_neg}")
print(f"Absolute value: {np.abs(arr_neg)}")
print(f"Sign: {np.sign(arr_neg)}")

# Modulo and remainder
print(f"\n=== MODULO AND REMAINDER ===")
print(f"Array: {arr}")
print(f"Modulo 3: {np.mod(arr, 3)}")
print(f"Remainder when divided by 3: {np.remainder(arr, 3)}")
print(f"Floating point remainder: {np.fmod(np.array([5.0, 3.0, 2.0]), 2.0

# Special functions
print(f"\n=== SPECIAL FUNCTIONS ===")
```

```
print(f"Factorial of 5: {np.math.factorial(5)}")
print(f"Gamma function: {np.array([np.math.gamma(x) for x in [1, 2, 3, 4,
print(f"Beta function B(2,3): {np.math.beta(2, 3)}")

# Complex number functions
complex_arr = np.array([1+2j, 3+4j, 5+6j])
print(f"\n=== COMPLEX NUMBER FUNCTIONS ===")
print(f"Complex array: {complex_arr}")
print(f"Real part: {np.real(complex_arr)}")
print(f"Imaginary part: {np.imag(complex_arr)}")
print(f"Magnitude: {np.abs(complex_arr)}")
print(f"Phase angle: {np.angle(complex_arr)}")
print(f"Conjugate: {np.conj(complex_arr)}")
```

7. Statistical Operations

NumPy provides comprehensive statistical functions for analyzing data distributions, calculating descriptive statistics, and performing various statistical tests.

In [ ]:
```
# Statistical Operations Examples

# Create sample datasets
np.random.seed(42)
data = np.random.normal(100, 15, 1000)  # Normal distribution
data_2d = np.random.normal(50, 10, (5, 4))  # 2D normal data
data_with_outliers = np.array([1, 2, 3, 4, 5, 100, 6, 7, 8, 9])  # With o

print("=== DESCRIPTIVE STATISTICS ===")
print(f"Sample data (first 10): {data[:10]}")
print(f"Data shape: {data.shape}")

# Central tendency measures
print(f"\n--- CENTRAL TENDENCY ---")
print(f"Mean: {np.mean(data):.2f}")
print(f"Median: {np.median(data):.2f}")
print(f"Mode (most frequent): {np.bincount(data.astype(int)).argmax()}")
print(f"Geometric mean: {np.exp(np.mean(np.log(data[data > 0]))):.2f}")
print(f"Harmonic mean: {len(data) / np.sum(1/data[data > 0]):.2f}")

# Dispersion measures
print(f"\n--- DISPERSION MEASURES ---")
print(f"Standard deviation: {np.std(data):.2f}")
print(f"Variance: {np.var(data):.2f}")
print(f"Range: {np.ptp(data):.2f}")  # Peak-to-peak (max - min)
print(f"Interquartile range: {np.percentile(data, 75) - np.percentile(dat
print(f"Mean absolute deviation: {np.mean(np.abs(data - np.mean(data))):.

# Percentiles and quantiles
print(f"\n--- PERCENTILES AND QUANTILES ---")
percentiles = [0, 25, 50, 75, 100]
for p in percentiles:
    print(f"{p}th percentile: {np.percentile(data, p):.2f}")

# Quantiles
quantiles = np.quantile(data, [0.1, 0.25, 0.5, 0.75, 0.9])
print(f"Quantiles [0.1, 0.25, 0.5, 0.75, 0.9]: {quantiles}")

# Shape measures
```

```python
print(f"\n--- SHAPE MEASURES ---")
from scipy import stats
print(f"Skewness: {stats.skew(data):.3f}")
print(f"Kurtosis: {stats.kurtosis(data):.3f}")

# 2D array statistics
print(f"\n=== 2D ARRAY STATISTICS ===")
print(f"2D data:\n{data_2d}")

# Axis-wise statistics
print(f"\n--- AXIS-WISE STATISTICS ---")
print(f"Mean along axis 0 (columns): {np.mean(data_2d, axis=0)}")
print(f"Mean along axis 1 (rows): {np.mean(data_2d, axis=1)}")
print(f"Std along axis 0: {np.std(data_2d, axis=0)}")
print(f"Min along axis 0: {np.min(data_2d, axis=0)}")
print(f"Max along axis 0: {np.max(data_2d, axis=0)}")

# Outlier detection
print(f"\n=== OUTLIER DETECTION ===")
print(f"Data with outliers: {data_with_outliers}")

# IQR method for outlier detection
Q1 = np.percentile(data_with_outliers, 25)
Q3 = np.percentile(data_with_outliers, 75)
IQR = Q3 - Q1
lower_bound = Q1 - 1.5 * IQR
upper_bound = Q3 + 1.5 * IQR

outliers = data_with_outliers[(data_with_outliers < lower_bound) | (data_
print(f"Q1: {Q1}, Q3: {Q3}, IQR: {IQR}")
print(f"Outlier bounds: [{lower_bound}, {upper_bound}]")
print(f"Outliers detected: {outliers}")

# Z-score method
z_scores = np.abs(stats.zscore(data_with_outliers))
outliers_z = data_with_outliers[z_scores > 2]
print(f"Outliers (Z-score > 2): {outliers_z}")

# Correlation and covariance
print(f"\n=== CORRELATION AND COVARIANCE ===")
x = np.array([1, 2, 3, 4, 5])
y = np.array([2, 4, 6, 8, 10])
print(f"X: {x}")
print(f"Y: {y}")
print(f"Correlation coefficient: {np.corrcoef(x, y)[0, 1]:.3f}")
print(f"Covariance: {np.cov(x, y)[0, 1]:.3f}")

# Histogram and binning
print(f"\n=== HISTOGRAM AND BINNING ===")
hist, bin_edges = np.histogram(data, bins=10)
print(f"Histogram counts: {hist}")
print(f"Bin edges: {bin_edges}")

# Digitize (assign values to bins)
values = np.array([85, 95, 105, 115, 125])
bin_indices = np.digitize(values, bin_edges)
print(f"Values: {values}")
print(f"Bin indices: {bin_indices}")

# Weighted statistics
```

```
weights = np.array([0.1, 0.2, 0.3, 0.4])
values_weighted = np.array([1, 2, 3, 4])
print(f"\n--- WEIGHTED STATISTICS ---")
print(f"Values: {values_weighted}")
print(f"Weights: {weights}")
print(f"Weighted mean: {np.average(values_weighted, weights=weights):.2f}
print(f"Weighted variance: {np.average((values_weighted - np.average(valu

# Moving averages
print(f"\n=== MOVING AVERAGES ===")
time_series = np.array([1, 2, 3, 4, 5, 6, 7, 8, 9, 10])
window = 3
moving_avg = np.convolve(time_series, np.ones(window)/window, mode='valid
print(f"Time series: {time_series}")
print(f"Moving average (window=3): {moving_avg}")
```

8. Linear Algebra Operations

NumPy provides comprehensive linear algebra functions through the `numpy.linalg` module, including matrix operations, eigenvalue decomposition, and solving linear systems.

In [ ]:
```
# Linear Algebra Operations Examples

# Create sample matrices
A = np.array([[1, 2, 3], [4, 5, 6], [7, 8, 9]])
B = np.array([[9, 8, 7], [6, 5, 4], [3, 2, 1]])
C = np.array([[2, 1], [1, 3]])
D = np.array([[1, 2, 3], [4, 5, 6]])  # 2x3 matrix
E = np.array([[1, 2], [3, 4], [5, 6]])  # 3x2 matrix

print("=== MATRIX OPERATIONS ===")
print(f"Matrix A:\n{A}")
print(f"Matrix B:\n{B}")

# Basic matrix operations
print(f"\n--- BASIC MATRIX OPERATIONS ---")
print(f"Matrix addition A + B:\n{A + B}")
print(f"Matrix subtraction A - B:\n{A - B}")
print(f"Element-wise multiplication A * B:\n{A * B}")
print(f"Matrix multiplication A @ B:\n{A @ B}")

# Matrix properties
print(f"\n--- MATRIX PROPERTIES ---")
print(f"Matrix A shape: {A.shape}")
print(f"Matrix A size: {A.size}")
print(f"Matrix A rank: {np.linalg.matrix_rank(A)}")
print(f"Matrix A trace: {np.trace(A)}")
print(f"Matrix A determinant: {np.linalg.det(C):.2f}")

# Transpose
print(f"\n--- TRANSPOSE ---")
print(f"Matrix A transpose:\n{A.T}")
print(f"Matrix D (2x3):\n{D}")
print(f"Matrix D transpose (3x2):\n{D.T}")

# Matrix powers
```

```python
print(f"\n--- MATRIX POWERS ---")
print(f"Matrix C squared:\n{np.linalg.matrix_power(C, 2)}")
print(f"Matrix C cubed:\n{np.linalg.matrix_power(C, 3)}")

# Inverse and pseudo-inverse
print(f"\n--- INVERSE AND PSEUDO-INVERSE ---")
try:
    C_inv = np.linalg.inv(C)
    print(f"Matrix C inverse:\n{C_inv}")
    print(f"C @ C_inv (should be identity):\n{C @ C_inv}")
except np.linalg.LinAlgError:
    print("Matrix C is singular (not invertible)")

# Pseudo-inverse for non-square matrices
D_pinv = np.linalg.pinv(D)
print(f"Matrix D pseudo-inverse:\n{D_pinv}")

# Solving linear systems
print(f"\n--- SOLVING LINEAR SYSTEMS ---")
# Ax = b
A_system = np.array([[3, 1], [1, 2]])
b = np.array([9, 8])
x = np.linalg.solve(A_system, b)
print(f"System: A = \n{A_system}")
print(f"Right-hand side b = {b}")
print(f"Solution x = {x}")
print(f"Verification A @ x = {A_system @ x}")

# Eigenvalues and eigenvectors
print(f"\n--- EIGENVALUES AND EIGENVECTORS ---")
eigenvals, eigenvecs = np.linalg.eig(C)
print(f"Matrix C:\n{C}")
print(f"Eigenvalues: {eigenvals}")
print(f"Eigenvectors:\n{eigenvecs}")

# Verify: A @ v = λ @ v
for i in range(len(eigenvals)):
    v = eigenvecs[:, i]
    λ = eigenvals[i]
    print(f"Eigenvalue {i+1}: {λ:.3f}")
    print(f"Verification: A @ v = {C @ v}")
    print(f"λ @ v = {λ * v}")
    print(f"Are they equal? {np.allclose(C @ v, λ * v)}")

# Singular Value Decomposition (SVD)
print(f"\n--- SINGULAR VALUE DECOMPOSITION ---")
U, s, Vt = np.linalg.svd(A)
print(f"Matrix A:\n{A}")
print(f"U matrix:\n{U}")
print(f"Singular values: {s}")
print(f"V^T matrix:\n{Vt}")

# Reconstruct original matrix
A_reconstructed = U @ np.diag(s) @ Vt
print(f"Reconstructed A:\n{A_reconstructed}")
print(f"Reconstruction error: {np.linalg.norm(A - A_reconstructed)}")

# QR decomposition
print(f"\n--- QR DECOMPOSITION ---")
Q, R = np.linalg.qr(C)
```

```
print(f"Matrix C:\n{C}")
print(f"Q matrix (orthogonal):\n{Q}")
print(f"R matrix (upper triangular):\n{R}")
print(f"Q @ R (should equal C):\n{Q @ R}")

# Cholesky decomposition (for positive definite matrices)
print(f"\n--- CHOLESKY DECOMPOSITION ---")
# Create a positive definite matrix
P = C @ C.T  # This will be positive definite
L = np.linalg.cholesky(P)
print(f"Positive definite matrix P:\n{P}")
print(f"Cholesky factor L:\n{L}")
print(f"L @ L.T (should equal P):\n{L @ L.T}")

# Matrix norms
print(f"\n--- MATRIX NORMS ---")
print(f"Frobenius norm of A: {np.linalg.norm(A, 'fro'):.3f}")
print(f"L2 norm of A: {np.linalg.norm(A, 2):.3f}")
print(f"L1 norm of A: {np.linalg.norm(A, 1):.3f}")
print(f"Infinity norm of A: {np.linalg.norm(A, np.inf):.3f}")

# Vector operations
print(f"\n--- VECTOR OPERATIONS ---")
v1 = np.array([1, 2, 3])
v2 = np.array([4, 5, 6])
print(f"Vector v1: {v1}")
print(f"Vector v2: {v2}")
print(f"Dot product: {np.dot(v1, v2)}")
print(f"Cross product: {np.cross(v1, v2)}")
print(f"Vector norm: {np.linalg.norm(v1):.3f}")
print(f"Unit vector: {v1 / np.linalg.norm(v1)}")

# Angle between vectors
cos_angle = np.dot(v1, v2) / (np.linalg.norm(v1) * np.linalg.norm(v2))
angle_rad = np.arccos(cos_angle)
angle_deg = np.degrees(angle_rad)
print(f"Angle between v1 and v2: {angle_deg:.2f} degrees")

# Projection
proj_v2_on_v1 = (np.dot(v1, v2) / np.dot(v1, v1)) * v1
print(f"Projection of v2 onto v1: {proj_v2_on_v1}")

# Matrix condition number
print(f"\n--- MATRIX CONDITION NUMBER ---")
cond_num = np.linalg.cond(C)
print(f"Condition number of C: {cond_num:.3f}")
if cond_num < 1e12:
    print("Matrix is well-conditioned")
else:
    print("Matrix is ill-conditioned")
```

9. Array Reshaping and Manipulation

NumPy provides powerful functions for reshaping, splitting, joining, and manipulating arrays to prepare data for analysis and visualization.

In [ ]:
```
# Array Reshaping and Manipulation Examples
```

```python
# Create sample arrays
arr_1d = np.arange(12)
arr_2d = np.array([[1, 2, 3], [4, 5, 6], [7, 8, 9]])
arr_3d = np.arange(24).reshape(2, 3, 4)

print("=== ARRAY RESHAPING ===")
print(f"Original 1D array: {arr_1d}")
print(f"Shape: {arr_1d.shape}")

# Basic reshaping
print(f"\n--- BASIC RESHAPING ---")
reshaped_2d = arr_1d.reshape(3, 4)
print(f"Reshaped to (3, 4):\n{reshaped_2d}")

reshaped_3d = arr_1d.reshape(2, 2, 3)
print(f"Reshaped to (2, 2, 3):\n{reshaped_3d}")

# Flattening
print(f"\n--- FLATTENING ---")
print(f"Original 2D array:\n{arr_2d}")
print(f"Flattened (C-order): {arr_2d.flatten()}")
print(f"Flattened (F-order): {arr_2d.flatten('F')}")
print(f"Ravel (C-order): {arr_2d.ravel()}")
print(f"Ravel (F-order): {arr_2d.ravel('F')}")

# Transpose and axes
print(f"\n--- TRANSPOSE AND AXES ---")
print(f"Original 3D array shape: {arr_3d.shape}")
print(f"Transpose (reverse axes): {arr_3d.T.shape}")
print(f"Transpose specific axes (0,2,1): {arr_3d.transpose(0, 2, 1).shape

# Resize vs reshape
print(f"\n--- RESIZE VS RESHAPE ---")
arr_small = np.array([1, 2, 3, 4])
print(f"Original array: {arr_small}")

# Resize (modifies original array)
arr_resized = arr_small.copy()
arr_resized.resize(6)
print(f"Resized to 6 elements: {arr_resized}")

# Reshape (creates new array)
arr_reshaped = arr_small.reshape(2, 2)
print(f"Reshaped to (2, 2):\n{arr_reshaped}")

# Array splitting
print(f"\n=== ARRAY SPLITTING ===")
arr_to_split = np.arange(12)
print(f"Array to split: {arr_to_split}")

# Split into equal parts
split_arrays = np.split(arr_to_split, 3)
print(f"Split into 3 equal parts:")
for i, arr in enumerate(split_arrays):
    print(f"  Part {i+1}: {arr}")

# Split at specific indices
split_at_indices = np.split(arr_to_split, [3, 7])
print(f"Split at indices [3, 7]:")
for i, arr in enumerate(split_at_indices):
```

```
    print(f"  Part {i+1}: {arr}")

# Vertical and horizontal splitting
print(f"\n--- VERTICAL AND HORIZONTAL SPLITTING ---")
print(f"2D array:\n{arr_2d}")

# Vertical split (split rows)
vsplit_arrays = np.vsplit(arr_2d, 3)
print(f"Vertical split (by rows):")
for i, arr in enumerate(vsplit_arrays):
    print(f"  Row {i+1}:\n{arr}")

# Horizontal split (split columns)
hsplit_arrays = np.hsplit(arr_2d, 3)
print(f"Horizontal split (by columns):")
for i, arr in enumerate(hsplit_arrays):
    print(f"  Column {i+1}:\n{arr}")

# Array joining
print(f"\n=== ARRAY JOINING ===")
arr1 = np.array([1, 2, 3])
arr2 = np.array([4, 5, 6])
arr3 = np.array([7, 8, 9])

print(f"Array 1: {arr1}")
print(f"Array 2: {arr2}")
print(f"Array 3: {arr3}")

# Concatenate
print(f"\n--- CONCATENATION ---")
concatenated = np.concatenate([arr1, arr2, arr3])
print(f"Concatenated: {concatenated}")

# Stack (creates new dimension)
stacked = np.stack([arr1, arr2, arr3])
print(f"Stacked (creates new axis):\n{stacked}")
print(f"Stacked shape: {stacked.shape}")

# Vstack and hstack
vstacked = np.vstack([arr1, arr2, arr3])
hstacked = np.hstack([arr1, arr2, arr3])
print(f"V-stacked:\n{vstacked}")
print(f"H-stacked: {hstacked}")

# Column and row stacking
col_stacked = np.column_stack([arr1, arr2, arr3])
row_stacked = np.row_stack([arr1, arr2, arr3])
print(f"Column-stacked:\n{col_stacked}")
print(f"Row-stacked:\n{row_stacked}")

# Array manipulation
print(f"\n=== ARRAY MANIPULATION ===")

# Repeat elements
print(f"\n--- REPEATING ELEMENTS ---")
arr_repeat = np.array([1, 2, 3])
print(f"Original: {arr_repeat}")
print(f"Repeat each element 3 times: {np.repeat(arr_repeat, 3)}")
print(f"Repeat entire array 2 times: {np.tile(arr_repeat, 2)}")
```

```python
# Insert and delete
print(f"\n--- INSERT AND DELETE ---")
arr_insert = np.array([1, 2, 4, 5])
print(f"Original: {arr_insert}")

# Insert element
arr_with_insert = np.insert(arr_insert, 2, 3)
print(f"Insert 3 at index 2: {arr_with_insert}")

# Delete element
arr_with_delete = np.delete(arr_with_insert, 1)
print(f"Delete element at index 1: {arr_with_delete}")

# Append
print(f"\n--- APPEND ---")
arr_append = np.append(arr_insert, [6, 7, 8])
print(f"Append [6, 7, 8]: {arr_append}")

# Array sorting
print(f"\n=== ARRAY SORTING ===")
arr_unsorted = np.array([3, 1, 4, 1, 5, 9, 2, 6])
print(f"Unsorted array: {arr_unsorted}")

# Sort
arr_sorted = np.sort(arr_unsorted)
print(f"Sorted array: {arr_sorted}")

# Sort in-place
arr_inplace = arr_unsorted.copy()
arr_inplace.sort()
print(f"Sorted in-place: {arr_inplace}")

# Argsort (indices that would sort)
sort_indices = np.argsort(arr_unsorted)
print(f"Sort indices: {sort_indices}")
print(f"Array sorted by indices: {arr_unsorted[sort_indices]}")

# Sort along specific axis
arr_2d_unsorted = np.array([[3, 1, 4], [1, 5, 9], [2, 6, 5]])
print(f"\n2D unsorted array:\n{arr_2d_unsorted}")
print(f"Sorted along axis 0 (columns):\n{np.sort(arr_2d_unsorted, axis=0)
print(f"Sorted along axis 1 (rows):\n{np.sort(arr_2d_unsorted, axis=1)}")

# Unique elements
print(f"\n=== UNIQUE ELEMENTS ===")
arr_with_duplicates = np.array([1, 2, 2, 3, 3, 3, 4, 5])
print(f"Array with duplicates: {arr_with_duplicates}")

unique_elements = np.unique(arr_with_duplicates)
print(f"Unique elements: {unique_elements}")

# Unique with counts
unique_elements, counts = np.unique(arr_with_duplicates, return_counts=Tr
print(f"Unique elements: {unique_elements}")
print(f"Counts: {counts}")

# Unique with indices
unique_elements, indices = np.unique(arr_with_duplicates, return_index=Tr
print(f"Unique elements: {unique_elements}")
print(f"First occurrence indices: {indices}")
```

```
# Array padding
print(f"\n=== ARRAY PADDING ===")
arr_pad = np.array([1, 2, 3])
print(f"Original array: {arr_pad}")

# Pad with zeros
padded_zeros = np.pad(arr_pad, (1, 2), mode='constant', constant_values=0
print(f"Padded with zeros: {padded_zeros}")

# Pad with edge values
padded_edge = np.pad(arr_pad, (1, 1), mode='edge')
print(f"Padded with edge values: {padded_edge}")

# Pad with reflection
padded_reflect = np.pad(arr_pad, (1, 1), mode='reflect')
print(f"Padded with reflection: {padded_reflect}")

# Array broadcasting for reshaping
print(f"\n=== BROADCASTING FOR RESHAPING ===")
# Create arrays that can be broadcast together
arr_a = np.array([[1], [2], [3]])  # Shape (3, 1)
arr_b = np.array([10, 20, 30])     # Shape (3,)

print(f"Array A shape: {arr_a.shape}")
print(f"Array B shape: {arr_b.shape}")
print(f"Broadcasted result shape: {(arr_a + arr_b).shape}")
print(f"Broadcasted result:\n{arr_a + arr_b}")

# Meshgrid for coordinate arrays
print(f"\n=== MESHGRID FOR COORDINATE ARRAYS ===")
x = np.array([1, 2, 3])
y = np.array([10, 20])
X, Y = np.meshgrid(x, y)
print(f"X coordinates:\n{X}")
print(f"Y coordinates:\n{Y}")
print(f"Combined coordinates:")
for i in range(len(y)):
    for j in range(len(x)):
        print(f"  ({X[i,j]}, {Y[i,j]})")
```

10. Working with Missing Data

NumPy provides several ways to handle missing data, including NaN (Not a Number) values and masked arrays. Understanding how to work with missing data is crucial for real-world data analysis.

In [ ]:
```
# Working with Missing Data Examples

import numpy as np
import numpy.ma as ma

print("=== WORKING WITH NaN VALUES ===")

# Creating arrays with NaN values
arr_with_nan = np.array([1, 2, np.nan, 4, 5])
print(f"Array with NaN: {arr_with_nan}")
```

```
print(f"Data type: {arr_with_nan.dtype}")

# Checking for NaN values
print(f"\n--- CHECKING FOR NaN VALUES ---")
print(f"Array: {arr_with_nan}")
print(f"isnan(): {np.isnan(arr_with_nan)}")
print(f"isnan() any: {np.isnan(arr_with_nan).any()}")
print(f"isnan() all: {np.isnan(arr_with_nan).all()}")
print(f"Number of NaN values: {np.isnan(arr_with_nan).sum()}")

# Operations with NaN
print(f"\n--- OPERATIONS WITH NaN ---")
print(f"Sum with NaN: {np.sum(arr_with_nan)}")
print(f"Mean with NaN: {np.mean(arr_with_nan)}")
print(f"Sum ignoring NaN: {np.nansum(arr_with_nan)}")
print(f"Mean ignoring NaN: {np.nanmean(arr_with_nan)}")
print(f"Standard deviation ignoring NaN: {np.nanstd(arr_with_nan)}")

# Replacing NaN values
print(f"\n--- REPLACING NaN VALUES ---")
arr_no_nan = arr_with_nan.copy()
arr_no_nan[np.isnan(arr_no_nan)] = 0
print(f"Replace NaN with 0: {arr_no_nan}")

# Replace with mean
arr_mean_filled = arr_with_nan.copy()
mean_value = np.nanmean(arr_with_nan)
arr_mean_filled[np.isnan(arr_mean_filled)] = mean_value
print(f"Replace NaN with mean ({mean_value:.2f}): {arr_mean_filled}")

# Interpolation for NaN values
print(f"\n--- INTERPOLATION FOR NaN VALUES ---")
arr_interp = np.array([1, 2, np.nan, np.nan, 5, 6])
print(f"Original array: {arr_interp}")

# Forward fill
arr_ffill = arr_interp.copy()
mask = np.isnan(arr_ffill)
arr_ffill[mask] = np.interp(np.flatnonzero(mask), np.flatnonzero(~mask),
print(f"Forward filled: {arr_ffill}")

# Backward fill
arr_bfill = arr_interp.copy()
mask = np.isnan(arr_bfill)
arr_bfill[mask] = np.interp(np.flatnonzero(mask), np.flatnonzero(~mask),
print(f"Backward filled: {arr_bfill}")

print(f"\n=== MASKED ARRAYS ===")

# Creating masked arrays
data = np.array([1, 2, 3, 4, 5, 6, 7, 8, 9, 10])
mask = np.array([False, False, True, False, True, False, False, True, Fal
masked_arr = ma.masked_array(data, mask=mask)

print(f"Original data: {data}")
print(f"Mask: {mask}")
print(f"Masked array: {masked_arr}")
print(f"Compressed (valid data): {masked_arr.compressed()}")
print(f"Masked values: {masked_arr[masked_arr.mask]}")
```

```python
# Operations on masked arrays
print(f"\n--- OPERATIONS ON MASKED ARRAYS ---")
print(f"Sum: {masked_arr.sum()}")
print(f"Mean: {masked_arr.mean():.2f}")
print(f"Standard deviation: {masked_arr.std():.2f}")
print(f"Min: {masked_arr.min()}")
print(f"Max: {masked_arr.max()}")

# Creating masked arrays from conditions
print(f"\n--- CREATING MASKED ARRAYS FROM CONDITIONS ---")
arr_condition = np.array([1, 5, 3, 8, 2, 9, 4, 7, 6])
masked_condition = ma.masked_where(arr_condition < 5, arr_condition)
print(f"Original: {arr_condition}")
print(f"Masked where < 5: {masked_condition}")

# Masked arrays with NaN
print(f"\n--- MASKED ARRAYS WITH NaN ---")
arr_nan_mask = np.array([1, 2, np.nan, 4, 5, np.nan, 7])
masked_nan = ma.masked_invalid(arr_nan_mask)
print(f"Original with NaN: {arr_nan_mask}")
print(f"Masked invalid: {masked_nan}")
print(f"Valid data: {masked_nan.compressed()}")

# Filling masked values
print(f"\n--- FILLING MASKED VALUES ---")
filled_arr = masked_arr.filled(0)
print(f"Filled with 0: {filled_arr}")

filled_mean = masked_arr.filled(masked_arr.mean())
print(f"Filled with mean: {filled_mean}")

# 2D masked arrays
print(f"\n=== 2D MASKED ARRAYS ===")
data_2d = np.array([[1, 2, 3], [4, 5, 6], [7, 8, 9]])
mask_2d = np.array([[False, True, False], [True, False, False], [False, F
masked_2d = ma.masked_array(data_2d, mask=mask_2d)

print(f"2D data:\n{data_2d}")
print(f"2D mask:\n{mask_2d}")
print(f"2D masked array:\n{masked_2d}")

# Statistics on 2D masked arrays
print(f"\n--- 2D MASKED ARRAY STATISTICS ---")
print(f"Sum: {masked_2d.sum()}")
print(f"Mean: {masked_2d.mean():.2f}")
print(f"Sum along axis 0: {masked_2d.sum(axis=0)}")
print(f"Mean along axis 1: {masked_2d.mean(axis=1)}")

# Working with infinity
print(f"\n=== WORKING WITH INFINITY ===")
arr_inf = np.array([1, 2, np.inf, 4, -np.inf, 6])
print(f"Array with infinity: {arr_inf}")
print(f"isinf(): {np.isinf(arr_inf)}")
print(f"isfinite(): {np.isfinite(arr_inf)}")
print(f"isposinf(): {np.isposinf(arr_inf)}")
print(f"isneginf(): {np.isneginf(arr_inf)}")

# Replace infinity with NaN
arr_inf_to_nan = arr_inf.copy()
arr_inf_to_nan[np.isinf(arr_inf_to_nan)] = np.nan
```

```
print(f"Replace inf with NaN: {arr_inf_to_nan}")

# Statistical functions that handle NaN
print(f"\n=== STATISTICAL FUNCTIONS WITH NaN ===")
arr_mixed = np.array([1, 2, np.nan, 4, 5, np.inf, 7, 8])
print(f"Mixed array: {arr_mixed}")

# Using nan functions
print(f"nansum: {np.nansum(arr_mixed)}")
print(f"nanmean: {np.nanmean(arr_mixed):.2f}")
print(f"nanstd: {np.nanstd(arr_mixed):.2f}")
print(f"nanmin: {np.nanmin(arr_mixed)}")
print(f"nanmax: {np.nanmax(arr_mixed)}")

# Percentile with NaN
print(f"nanpercentile (50th): {np.nanpercentile(arr_mixed, 50):.2f}")

# Correlation with NaN
print(f"\n=== CORRELATION WITH NaN ===")
x_with_nan = np.array([1, 2, np.nan, 4, 5])
y_with_nan = np.array([2, 4, 6, np.nan, 10])

# Remove NaN pairs for correlation
mask = ~(np.isnan(x_with_nan) | np.isnan(y_with_nan))
x_clean = x_with_nan[mask]
y_clean = y_with_nan[mask]

print(f"X with NaN: {x_with_nan}")
print(f"Y with NaN: {y_with_nan}")
print(f"Clean X: {x_clean}")
print(f"Clean Y: {y_clean}")
print(f"Correlation: {np.corrcoef(x_clean, y_clean)[0, 1]:.3f}")

# Using masked arrays for correlation
x_masked = ma.masked_invalid(x_with_nan)
y_masked = ma.masked_invalid(y_with_nan)
correlation = ma.corrcoef(x_masked, y_masked)[0, 1]
print(f"Correlation with masked arrays: {correlation:.3f}")
```

11. Performance Comparison with Python Lists

Understanding the performance differences between NumPy arrays and Python lists is crucial for writing efficient numerical code. This section provides comprehensive benchmarks and explains why NumPy is faster.

In [ ]:
```
# Performance Comparison Examples

import numpy as np
import time
import sys
import matplotlib.pyplot as plt

print("=== PERFORMANCE COMPARISON: NUMPY vs PYTHON LISTS ===")

# Test different array sizes
sizes = [1000, 10000, 100000, 1000000]
operations = ['creation', 'addition', 'multiplication', 'sum', 'mean']
```

```python
# Store results
results = {op: {'numpy': [], 'python': []} for op in operations}

print(f"{'Size':<10} {'Operation':<15} {'NumPy (ms)':<12} {'Python (ms)':
print("-" * 70)

for size in sizes:
    # Array creation
    start = time.time()
    np_arr = np.arange(size)
    numpy_time = (time.time() - start) * 1000

    start = time.time()
    py_list = list(range(size))
    python_time = (time.time() - start) * 1000

    results['creation']['numpy'].append(numpy_time)
    results['creation']['python'].append(python_time)
    speedup = python_time / numpy_time if numpy_time > 0 else 0
    print(f"{size:<10} {'Creation':<15} {numpy_time:<12.3f} {python_time:

    # Element-wise addition
    np_arr2 = np.arange(size)
    py_list2 = list(range(size))

    start = time.time()
    np_result = np_arr + np_arr2
    numpy_time = (time.time() - start) * 1000

    start = time.time()
    py_result = [a + b for a, b in zip(py_list, py_list2)]
    python_time = (time.time() - start) * 1000

    results['addition']['numpy'].append(numpy_time)
    results['addition']['python'].append(python_time)
    speedup = python_time / numpy_time if numpy_time > 0 else 0
    print(f"{size:<10} {'Addition':<15} {numpy_time:<12.3f} {python_time:

    # Element-wise multiplication
    start = time.time()
    np_result = np_arr * 2
    numpy_time = (time.time() - start) * 1000

    start = time.time()
    py_result = [x * 2 for x in py_list]
    python_time = (time.time() - start) * 1000

    results['multiplication']['numpy'].append(numpy_time)
    results['multiplication']['python'].append(python_time)
    speedup = python_time / numpy_time if numpy_time > 0 else 0
    print(f"{size:<10} {'Multiplication':<15} {numpy_time:<12.3f} {python

    # Sum operation
    start = time.time()
    np_sum = np.sum(np_arr)
    numpy_time = (time.time() - start) * 1000

    start = time.time()
    py_sum = sum(py_list)
```

```
    python_time = (time.time() - start) * 1000

    results['sum']['numpy'].append(numpy_time)
    results['sum']['python'].append(python_time)
    speedup = python_time / numpy_time if numpy_time > 0 else 0
    print(f"{size:<10} {'Sum':<15} {numpy_time:<12.3f} {python_time:<12.3

    # Mean operation
    start = time.time()
    np_mean = np.mean(np_arr)
    numpy_time = (time.time() - start) * 1000

    start = time.time()
    py_mean = sum(py_list) / len(py_list)
    python_time = (time.time() - start) * 1000

    results['mean']['numpy'].append(numpy_time)
    results['mean']['python'].append(python_time)
    speedup = python_time / numpy_time if numpy_time > 0 else 0
    print(f"{size:<10} {'Mean':<15} {numpy_time:<12.3f} {python_time:<12.

    print()

print("=== MEMORY USAGE COMPARISON ===")

# Memory usage comparison
sizes_memory = [1000, 10000, 100000]
print(f"{'Size':<10} {'NumPy (MB)':<12} {'Python (MB)':<12} {'Memory Rati
print("-" * 50)

for size in sizes_memory:
    # NumPy array
    np_arr = np.arange(size, dtype=np.int64)
    numpy_memory = np_arr.nbytes / (1024 * 1024)  # Convert to MB

    # Python list
    py_list = list(range(size))
    python_memory = sys.getsizeof(py_list) + sum(sys.getsizeof(x) for x i
    python_memory = python_memory / (1024 * 1024)  # Convert to MB

    ratio = python_memory / numpy_memory if numpy_memory > 0 else 0
    print(f"{size:<10} {numpy_memory:<12.3f} {python_memory:<12.3f} {rati

print(f"\n=== WHY NUMPY IS FASTER ===")
print("""
1. **C Implementation**: NumPy is implemented in C, which is much faster
2. **Vectorized Operations**: Operations are performed on entire arrays a
3. **Memory Layout**: Contiguous memory layout enables better cache utili
4. **No Type Checking**: NumPy arrays have fixed types, eliminating runti
5. **Optimized Libraries**: Uses optimized BLAS/LAPACK libraries for line
6. **Broadcasting**: Efficient handling of operations between arrays of d
7. **SIMD Instructions**: Can utilize Single Instruction, Multiple Data i
""")

print("=== ADVANCED PERFORMANCE TESTS ===")

# Matrix multiplication performance
print(f"\n--- MATRIX MULTIPLICATION PERFORMANCE ---")
matrix_sizes = [100, 500, 1000]
```

```python
for size in matrix_sizes:
    # NumPy matrix multiplication
    A = np.random.random((size, size))
    B = np.random.random((size, size))

    start = time.time()
    C = np.dot(A, B)
    numpy_time = time.time() - start

    # Python list matrix multiplication (much slower)
    A_list = A.tolist()
    B_list = B.tolist()

    start = time.time()
    C_list = [[sum(a * b for a, b in zip(row, col)) for col in zip(*B_lis
    python_time = time.time() - start

    speedup = python_time / numpy_time if numpy_time > 0 else 0
    print(f"Matrix size {size}x{size}: NumPy {numpy_time:.4f}s, Python {p

# Broadcasting performance
print(f"\n--- BROADCASTING PERFORMANCE ---")
size = 1000000
arr1 = np.random.random(size)
arr2 = np.random.random((100, size))

start = time.time()
result = arr1 + arr2  # Broadcasting
numpy_time = time.time() - start

# Equivalent Python code (much slower)
start = time.time()
result_py = [[a + b for a, b in zip(arr1, row)] for row in arr2]
python_time = time.time() - start

speedup = python_time / numpy_time if numpy_time > 0 else 0
print(f"Broadcasting {size} elements: NumPy {numpy_time:.4f}s, Python {py

# Universal functions performance
print(f"\n--- UNIVERSAL FUNCTIONS PERFORMANCE ---")
arr = np.random.random(1000000)

# NumPy ufuncs
start = time.time()
result = np.sin(arr) + np.cos(arr) + np.sqrt(arr)
numpy_time = time.time() - start

# Python equivalent
arr_list = arr.tolist()
start = time.time()
result_py = [math.sin(x) + math.cos(x) + math.sqrt(x) for x in arr_list]
python_time = time.time() - start

speedup = python_time / numpy_time if numpy_time > 0 else 0
print(f"Math functions on {len(arr)} elements: NumPy {numpy_time:.4f}s, P

print(f"\n=== PERFORMANCE TIPS ===")
print("""
1. **Use vectorized operations** instead of loops
2. **Avoid Python loops** when possible
```

```
3. **Use appropriate data types** (int32 vs int64)
4. **Pre-allocate arrays** when you know the size
5. **Use in-place operations** when possible (e.g., += instead of +)
6. **Avoid unnecessary copies** of arrays
7. **Use views instead of copies** when possible
8. **Profile your code** to identify bottlenecks
9. **Consider using numba** for additional speedup
10. **Use NumPy's built-in functions** instead of custom implementations
""")

# Memory efficiency example
print(f"\n=== MEMORY EFFICIENCY EXAMPLE ===")
size = 1000000

# NumPy array
np_arr = np.arange(size, dtype=np.int32)
print(f"NumPy int32 array: {np_arr.nbytes / (1024*1024):.2f} MB")

# Python list
py_list = list(range(size))
py_memory = sys.getsizeof(py_list) + sum(sys.getsizeof(x) for x in py_lis
print(f"Python list: {py_memory / (1024*1024):.2f} MB")

# Memory ratio
ratio = py_memory / np_arr.nbytes
print(f"Python uses {ratio:.1f}x more memory than NumPy")

# Data type optimization
print(f"\n--- DATA TYPE OPTIMIZATION ---")
arr_int64 = np.arange(1000000, dtype=np.int64)
arr_int32 = np.arange(1000000, dtype=np.int32)
arr_float64 = np.arange(1000000, dtype=np.float64)
arr_float32 = np.arange(1000000, dtype=np.float32)

print(f"int64: {arr_int64.nbytes / (1024*1024):.2f} MB")
print(f"int32: {arr_int32.nbytes / (1024*1024):.2f} MB")
print(f"float64: {arr_float64.nbytes / (1024*1024):.2f} MB")
print(f"float32: {arr_float32.nbytes / (1024*1024):.2f} MB")

# Performance with different data types
print(f"\n--- PERFORMANCE WITH DIFFERENT DATA TYPES ---")
arr_int32 = np.random.randint(0, 100, 1000000, dtype=np.int32)
arr_float32 = np.random.random(1000000).astype(np.float32)
arr_float64 = np.random.random(1000000).astype(np.float64)

start = time.time()
result = arr_int32 + arr_int32
int32_time = time.time() - start

start = time.time()
result = arr_float32 + arr_float32
float32_time = time.time() - start

start = time.time()
result = arr_float64 + arr_float64
float64_time = time.time() - start

print(f"int32 addition: {int32_time:.6f}s")
print(f"float32 addition: {float32_time:.6f}s")
print(f"float64 addition: {float64_time:.6f}s")
```

12. File I/O Operations

NumPy provides efficient functions for saving and loading arrays to/from files. This is essential for data persistence and sharing data between different programs and languages.

In [ ]:
```
# File I/O Operations Examples

import numpy as np
import os
import tempfile

print("=== NUMPY FILE I/O OPERATIONS ===")

# Create sample data
arr_1d = np.array([1, 2, 3, 4, 5])
arr_2d = np.array([[1, 2, 3], [4, 5, 6], [7, 8, 9]])
arr_3d = np.random.random((2, 3, 4))
arr_mixed = np.array([1.5, 2.7, np.nan, 4.2, 5.8])

print(f"Sample 1D array: {arr_1d}")
print(f"Sample 2D array:\n{arr_2d}")
print(f"Sample 3D array shape: {arr_3d.shape}")
print(f"Sample array with NaN: {arr_mixed}")

print(f"\n=== SAVING ARRAYS ===")

# 1. Save as .npy (NumPy binary format)
print(f"\n--- SAVING AS .NPY FORMAT ---")
np.save('array_1d.npy', arr_1d)
np.save('array_2d.npy', arr_2d)
np.save('array_3d.npy', arr_3d)
print("Arrays saved as .npy files")

# 2. Save multiple arrays as .npz (compressed)
print(f"\n--- SAVING AS .NPZ FORMAT ---")
np.savez('multiple_arrays.npz',
         array_1d=arr_1d,
         array_2d=arr_2d,
         array_3d=arr_3d,
         array_mixed=arr_mixed)
print("Multiple arrays saved as .npz file")

# 3. Save as compressed .npz
print(f"\n--- SAVING AS COMPRESSED .NPZ ---")
np.savez_compressed('compressed_arrays.npz',
                    array_1d=arr_1d,
                    array_2d=arr_2d,
                    array_3d=arr_3d,
                    array_mixed=arr_mixed)
print("Arrays saved as compressed .npz file")

# 4. Save as text files
print(f"\n--- SAVING AS TEXT FILES ---")
np.savetxt('array_1d.txt', arr_1d, fmt='%.2f')
np.savetxt('array_2d.txt', arr_2d, fmt='%.2f', delimiter=',')
print("Arrays saved as text files")
```

```python
# 5. Save with custom formatting
print(f"\n--- SAVING WITH CUSTOM FORMATTING ---")
np.savetxt('formatted_array.txt', arr_2d,
           fmt='%d',
           delimiter='\t',
           header='Row\tCol1\tCol2\tCol3',
           footer='End of data')
print("Array saved with custom formatting")

print(f"\n=== LOADING ARRAYS ===")

# 1. Load .npy files
print(f"\n--- LOADING .NPY FILES ---")
loaded_1d = np.load('array_1d.npy')
loaded_2d = np.load('array_2d.npy')
loaded_3d = np.load('array_3d.npy')

print(f"Loaded 1D array: {loaded_1d}")
print(f"Loaded 2D array:\n{loaded_2d}")
print(f"Loaded 3D array shape: {loaded_3d.shape}")
print(f"Arrays equal to original: {np.array_equal(arr_1d, loaded_1d)}")

# 2. Load .npz files
print(f"\n--- LOADING .NPZ FILES ---")
loaded_npz = np.load('multiple_arrays.npz')
print(f"Keys in .npz file: {list(loaded_npz.keys())}")

# Access individual arrays
loaded_1d_npz = loaded_npz['array_1d']
loaded_2d_npz = loaded_npz['array_2d']
loaded_3d_npz = loaded_npz['array_3d']
loaded_mixed_npz = loaded_npz['array_mixed']

print(f"Loaded 1D from .npz: {loaded_1d_npz}")
print(f"Loaded 2D from .npz:\n{loaded_2d_npz}")
print(f"Loaded mixed array from .npz: {loaded_mixed_npz}")

# Close the .npz file
loaded_npz.close()

# 3. Load compressed .npz
print(f"\n--- LOADING COMPRESSED .NPZ ---")
loaded_compressed = np.load('compressed_arrays.npz')
print(f"Keys in compressed .npz: {list(loaded_compressed.keys())}")
loaded_compressed.close()

# 4. Load text files
print(f"\n--- LOADING TEXT FILES ---")
loaded_txt_1d = np.loadtxt('array_1d.txt')
loaded_txt_2d = np.loadtxt('array_2d.txt', delimiter=',')

print(f"Loaded 1D from text: {loaded_txt_1d}")
print(f"Loaded 2D from text:\n{loaded_txt_2d}")

# 5. Load with custom parameters
print(f"\n--- LOADING WITH CUSTOM PARAMETERS ---")
loaded_formatted = np.loadtxt('formatted_array.txt',
                              delimiter='\t',
                              skiprows=1,  # Skip header
```

```
                                usecols=(1, 2, 3))  # Use only data columns
print(f"Loaded formatted array:\n{loaded_formatted}")

print(f"\n=== ADVANCED FILE I/O ===")

# Memory mapping for large arrays
print(f"\n--- MEMORY MAPPING ---")
# Create a large array
large_array = np.random.random((1000, 1000))
np.save('large_array.npy', large_array)

# Load as memory map
mmap_array = np.load('large_array.npy', mmap_mode='r')
print(f"Memory mapped array shape: {mmap_array.shape}")
print(f"Memory mapped array dtype: {mmap_array.dtype}")
print(f"Memory mapped array is read-only: {mmap_array.flags.writeable}")

# Access specific parts without loading entire array
print(f"First row: {mmap_array[0, :5]}")
print(f"First column: {mmap_array[:5, 0]}")

# Save with different data types
print(f"\n--- SAVING WITH DIFFERENT DATA TYPES ---")
arr_int32 = np.array([1, 2, 3, 4, 5], dtype=np.int32)
arr_float32 = np.array([1.1, 2.2, 3.3, 4.4, 5.5], dtype=np.float32)

np.save('int32_array.npy', arr_int32)
np.save('float32_array.npy', arr_float32)

# Load and check data types
loaded_int32 = np.load('int32_array.npy')
loaded_float32 = np.load('float32_array.npy')

print(f"Original int32: {arr_int32.dtype}")
print(f"Loaded int32: {loaded_int32.dtype}")
print(f"Original float32: {arr_float32.dtype}")
print(f"Loaded float32: {loaded_float32.dtype}")

# Save arrays with metadata
print(f"\n--- SAVING WITH METADATA ---")
# Create a structured array
structured_array = np.array([(1, 2.5, 'hello'), (3, 4.7, 'world')],
                            dtype=[('id', 'i4'), ('value', 'f4'), ('name',

np.save('structured_array.npy', structured_array)
loaded_structured = np.load('structured_array.npy')

print(f"Structured array: {structured_array}")
print(f"Loaded structured array: {loaded_structured}")
print(f"Field names: {structured_array.dtype.names}")

# Working with CSV files
print(f"\n--- WORKING WITH CSV FILES ---")
# Create sample data
data = np.array([[1, 2, 3, 4], [5, 6, 7, 8], [9, 10, 11, 12]])
np.savetxt('data.csv', data, delimiter=',', fmt='%d',
           header='col1,col2,col3,col4', comments='')

# Load CSV with column names
loaded_csv = np.loadtxt('data.csv', delimiter=',', skiprows=1)
```

```
print(f"Loaded CSV data:\n{loaded_csv}")

# Load specific columns
col1_col3 = np.loadtxt('data.csv', delimiter=',', skiprows=1, usecols=(0,
print(f"Columns 1 and 3:\n{col1_col3}")

# Working with different delimiters
print(f"\n--- WORKING WITH DIFFERENT DELIMITERS ---")
# Save with tab delimiter
np.savetxt('data.tsv', data, delimiter='\t', fmt='%d')
# Save with space delimiter
np.savetxt('data.space', data, delimiter=' ', fmt='%d')

# Load with different delimiters
loaded_tsv = np.loadtxt('data.tsv', delimiter='\t')
loaded_space = np.loadtxt('data.space', delimiter=' ')

print(f"TSV data:\n{loaded_tsv}")
print(f"Space-delimited data:\n{loaded_space}")

# Error handling
print(f"\n--- ERROR HANDLING ====")
try:
    # Try to load a non-existent file
    non_existent = np.load('non_existent.npy')
except FileNotFoundError as e:
    print(f"File not found error: {e}")

try:
    # Try to load with wrong delimiter
    wrong_delimiter = np.loadtxt('data.csv', delimiter=';')
except ValueError as e:
    print(f"Value error: {e}")

# File size comparison
print(f"\n--- FILE SIZE COMPARISON ---")
files = ['array_1d.npy', 'array_1d.txt', 'multiple_arrays.npz', 'compress
for file in files:
    if os.path.exists(file):
        size = os.path.getsize(file)
        print(f"{file}: {size} bytes")

# Clean up temporary files
print(f"\n--- CLEANING UP ---")
temp_files = ['array_1d.npy', 'array_2d.npy', 'array_3d.npy',
              'multiple_arrays.npz', 'compressed_arrays.npz',
              'array_1d.txt', 'array_2d.txt', 'formatted_array.txt',
              'large_array.npy', 'int32_array.npy', 'float32_array.npy',
              'structured_array.npy', 'data.csv', 'data.tsv', 'data.space

for file in temp_files:
    if os.path.exists(file):
        os.remove(file)
        print(f"Removed {file}")

print("All temporary files cleaned up!")

print(f"\n=== FILE I/O BEST PRACTICES ===")
print("""
1. **Use .npy for single arrays**: Fastest and most efficient
```

```
2. **Use .npz for multiple arrays**: Good for related data
3. **Use compressed .npz for large datasets**: Saves disk space
4. **Use memory mapping for very large arrays**: Avoids loading entire ar
5. **Use appropriate data types**: Saves memory and improves performance
6. **Include metadata when needed**: Use structured arrays or separate fi
7. **Handle errors gracefully**: Always use try-except blocks
8. **Consider file format compatibility**: .npy/.npz are NumPy-specific
9. **Use text formats for human readability**: CSV, TSV for data exchange
10. **Profile I/O operations**: Measure performance for large datasets
""")
```